

Margin-Based Classification

Margin-based classifiers define their loss functions in terms of the distance of the training data from the decision boundary (the margin). We will consider only binary classification. By placing the positive points as far away on the positive side, and the negative points as far away on the negative side, a margin-based classifier select a decision boundary that is exactly between the two classes. This is the basis for support vector machines (SVMs), the topic of study of this chapter, and an example of a margin-based classifier.

We start with linear classification. After establishing the solution for the linear case, we will use kernels to extend it to non-linear classification.

Mathematically, we will make a slight deviation from the previous (and subsequent) chapters. Before, it was simpler to include the intercept or constant term as w_0 and add an extra “feature” (always 1), called x_0 . However, for our derivations here, it will be simpler to explicitly name this extra term. Therefore, $\vec{x}$ will not have this extra feature. And, the discriminant will explicitly mention the constant term:

$$\tilde{f}(\vec{x}) = \vec{x}^T \vec{w} + b. \quad (7.1)$$

We are using b instead of w_0 to match the traditional notation for SVMs.

Margin

First, we need the formula of the distance of any particular point, $\vec{x}$, to the decision boundary. Figure 7.1 shows the geometry of the distance from $\vec{x}$ to the decision line (or plane or hyper-plane) $\vec{x}^T \vec{w} + b = 0$ and the resulting formula. The projection of the point $\vec{x}$ onto the direction of weight vector, $\vec{w}$, is $\frac{\vec{x}^T \vec{w}}{\|\vec{w}\|}$ because the projection is equal to the cosine of the angle between the vectors multiplied by the length of $\vec{x}$, and the inner product is equal to the same but also

Key point

Just for this chapter: Different notation for the constant term!

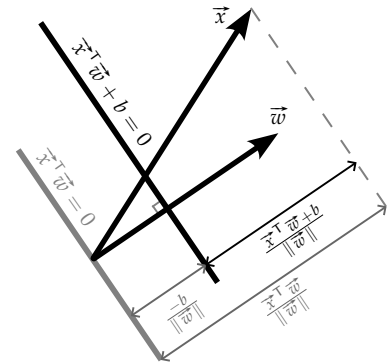


Figure 7.1: Geometry of the distance to the boundary. Note that b is negative in this diagram.

multiplied by the length of $\vec{w}$. A point is on the decision boundary when $\vec{x}^\top \vec{w} = -b$. Therefore, the same reasoning shows that the distance of the boundary to the origin is $\frac{-b}{\|\vec{w}\|}$. If we subtract these two, we get the formula for the distance of a data point $\vec{x}$ to the decision boundary:

$$\frac{\vec{x}^\top \vec{w} + b}{\|\vec{w}\|} = \frac{\vec{x}^\top \vec{w} + b}{\sqrt{\vec{w}^\top \vec{w}}} = \frac{\tilde{f}(\vec{x})}{\sqrt{\vec{w}^\top \vec{w}}} \quad (7.2) \quad \left\{ \begin{array}{l} \text{three ways of writing the same} \\ \text{thing} \end{array} \right.$$

This is just the value of the discriminant, divided by the magnitude of the weight vector (omitting the intercept term). It is a signed distance. It is positive for points on the positive side of the decision boundary and negative for points on the negative side.

We define the **margin of a data point** to be the signed distance to the decision boundary, but where the sign is positive if the point is correctly classified and negative if the point is incorrectly classified. To convert the distance above to a margin, we must multiply by the label associated with the point (+1 or -1). So, for training point $\vec{x}_i$, the margin is

$$\frac{y_i (\vec{x}_i^\top \vec{w} + b)}{\|\vec{w}\|} \quad (7.3)$$

This value is positive when example i is correctly classified and negative when it is not.

Equation 7.3 is the margin of a *point*. We define the **margin of a dataset** to be the smallest margin of any point in the dataset:

$$\min_{i=1}^m \frac{y_i (\vec{x}_i^\top \vec{w} + b)}{\|\vec{w}\|} \quad (7.4)$$

Figure 7.2 shows the margin of a particular dataset for a particular choice of $\vec{w}$ and b . It is the distance from the decision boundary to the nearest point, if all points are correctly classified.

The margins defined above (for a point and for a dataset) are called **geometric margins**. If we just consider the numerator, they are called **functional margins**, because they are the output of the discriminant function, multiplied by the correct label. These will be important later. So, in total we have four different definitions of a margin, depending on whether we are talking about a single point or an entire dataset and depending on whether we are talking about the geometric or functional margin. (See Table 7.3 for a summary.)

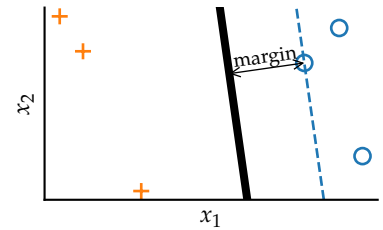


Figure 7.2: Margin of a dataset for a particular discriminant.

Maximum Margin Classification

A **support vector machine** (SVM) classifier picks the decision boundary that maximizes the geometric margin of the training data. Put differently, an SVM finds a boundary that is as far away from all

	Data point i	Dataset
Geometric	$\frac{y_i(\vec{x}_i^\top \vec{w} + b)}{\ \vec{w}\ }$	$\min_{i=1}^m \frac{y_i(\vec{x}_i^\top \vec{w} + b)}{\ \vec{w}\ }$
Functional	$y_i(\vec{x}_i^\top \vec{w} + b)$	$\min_{i=1}^m y_i(\vec{x}_i^\top \vec{w} + b)$

Table 7.3: Summary of different margin definitions.

training data as possible, while still correctly separating the training data into the two classes. For the moment, we will assume the two classes are separable. Later, we will extend the idea to non-separable data.

Figure 7.4 shows the maximum margin decision boundary for a simple dataset. Translating or rotating the decision boundary in any way will reduce the margin by making one of the points closer.

The data points that are exactly on the margin (three in the case of Figure 7.4) are called the **support vectors**. They *support* the final decision boundary: If any of them moved, the decision boundary would be different. Conversely, if any other point moved slightly, it would not change the decision boundary. So the boundary's position is supported by these particular data points.

To find the $\vec{w}$ and b that correspond to this point, we need to solve the minimization of Equation 7.4. We first note that if we scale both $\vec{w}$ and b by a positive quantity, the geometric margin does not change. Geometrically, the boundary does not move because the set of $\vec{x}$ vectors that satisfy $\vec{x}^\top \vec{w} + b = 0$ is the same if we multiply both sides by a positive scalar. Algebraically, the numerator and denominator in Equation 7.4 both scale by the same amount, so the fraction remains the same.

We will remove this extra degree of freedom. It would be natural to restrict $\vec{w}$ to be of unit length: $\|\vec{w}\| = 1$. However, for SVMs, it is more natural to restrict the numerator (the functional margin), $y_i(\vec{x}_i^\top \vec{w} + b)$. Specifically, if the training data are separable, we know all of the functional margins are positive. Therefore, we will set the smallest one to be equal to 1. This means that the geometric margin of the closest point (or points) is $\frac{1}{\|\vec{w}\|}$.

So, under the constraint that the functional margin is 1 for the closest point $(\vec{x}_i, y_i)$, maximizing the geometric margin is the same as minimizing $\|\vec{w}\|$ which is the same as minimizing $\|\vec{w}\|^2 = \vec{w}^\top \vec{w}$. This

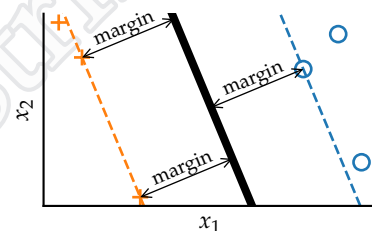


Figure 7.4: Maximum margin of a dataset.

leads us to the following constrained optimization problem:

both $\vec{w}$ and b
can be changed

$$\arg \min_{\vec{w}, b} \frac{1}{2} \vec{w}^\top \vec{w} \quad \text{--- objective function} \quad (7.5)$$

“such that” — (s.t.) $y_i (\vec{x}_i^\top \vec{w} + b) \geq 1$ for all $i \in \{1, \dots, m\}$ — one constraint for each data point

point’s functional margin must be at least 1

An optimization problem like this is called a (mathematical) program, which unrelated to the term “computer program.” It specifies four basic things:

1. what type of optimum we are looking for (minimum),
2. the function to be optimized¹ ($\frac{1}{2} \vec{w}^\top \vec{w}$),
3. the variables that can be changed ($\vec{w}$ and b), and
4. any constraints on the optimization ($y_i (\vec{x}_i^\top \vec{w} + b) \geq 1$).

¹ Called the objective function.

Because the objective function is a quadratic function (and the constraints are linear), this is called a **quadratic program** (or QP). The extra $\frac{1}{2}$ in the objective function makes the gradient a little nicer without changing the values of $\vec{w}$ and b that make it minimal. Note that while b does not appear in the objective function, it does still appear in the constraints and therefore must be considered.

Our goal was to minimize $\|\vec{w}\|$ while making sure that the functional margin was 1 for the closest point $(\vec{x}_i, y_i)$. The constraints above only ensure that the functional margin is greater than or equal to 1 for all points. However, when coupled with the objective of making $\vec{w}$ as small as possible, this achieves the same goal: If there were a $\vec{w}$ and b such that the functional margin is greater than 1 for all points, then there is slack in the constraints. Therefore, we can reduce $\vec{w}$ and b while still maintaining the functional margin constraints and get a better solution (one with a smaller objective). Therefore, any solution to this QP will have at least one data point with a functional margin equal to 1.

Solving a Quadratic Program

How to solve a QP numerically is beyond what we can cover. For QPs with linear inequality constraints and a convex objective function, general-purpose QP solvers take computational time that is at least cubic in the number of variables or the number of constraints. However, the QPs that arise from SVMs (the one above and those below) have special structure which can allow for faster solutions.

Therefore, unlike other chapters where we provide direct algorithms for learning, for SVMs no such pseudo-code is provided. Algorithm 7.5 demonstrates how to construct the matrices expected for a typical QP solver, but a solver specifically designed for an SVM

MAX-MARGIN LEARNING

In: $\mathbf{X}, \mathbf{Y}$

Out: $\vec{w}, b$

$$\mathbf{M} \leftarrow \begin{bmatrix} y_1 & \cdots & y_1 \vec{x}_1^\top & \cdots \\ y_2 & \cdots & y_2 \vec{x}_2^\top & \cdots \\ \vdots & & \vdots & \\ y_m & \cdots & y_m \vec{x}_m^\top & \cdots \end{bmatrix}$$

$$\tilde{\mathbf{I}} \leftarrow \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & & \\ 0 & 0 & 1 & & \\ & & & \ddots & \\ 0 & & & & 0 & 1 \end{bmatrix}$$

$$\vec{z} \leftarrow$$

$$\text{Solve-QP} \left(\begin{array}{l} \min \frac{1}{2} \vec{z}^\top \tilde{\mathbf{I}} \vec{z}, \\ \text{s.t. } \mathbf{M} \vec{z} \geq \vec{1} \end{array} \right)$$

$$\begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \leftarrow \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ \vdots \\ z_{n+1} \end{bmatrix}$$

return $\vec{w}, b$

Algorithm 7.5: Learning the maximum-margin binary classifier using a quadratic programming solver. The $n + 1$ -dimensional vector $\vec{z}$ comprises the variables of the QP: b followed by the elements of $\vec{w}$.

problem may work better than a general QP solver. Note that prediction (testing) is the same as other linear discriminant methods: Predict the label for input $\vec{x}$ to be the sign of $\vec{x}^\top \vec{w} + b$.

Non-Separable Data

While Equation 7.5 is fine for training data where the classes can be linearly separated, there are no solutions if the data are not separable. To allow for handling of non-separable data, we relax the constraints.

Specifically, we introduce a **slack variable**, ξ_i , for each data point. Instead of requiring the functional margin to be greater than 1 for this point, we require it to be greater than $1 - \xi_i$, relaxing the constraint. So, if $\xi_i < 1$ the point is still correctly classified, but is inside the margin. If $\xi_i > 1$, the point is on the wrong side of the decision boundary.

To keep the optimization from needlessly using the slack variables, we will add them to the objective function to penalize their use. Adding either their value or their square is possible, but it is more common to use just the absolute value. This yields the following QP:

$$\begin{aligned} \arg \min_{\vec{w}, b, \xi_1, \dots, \xi_m} \quad & \frac{1}{2} \vec{w}^\top \vec{w} + C \sum_{i=1}^m \xi_i \\ \text{s.t.} \quad & y_i (\vec{x}_i^\top \vec{w} + b) \geq 1 - \xi_i \quad \text{for all } i \in \{1, \dots, m\} \\ & \xi_i \geq 0 \quad \text{for all } i \in \{1, \dots, m\} \end{aligned} \quad (7.6)$$

This optimization problem is over the n -dimensional vector $\vec{w}$, the scalar b , and the m scalars $\xi_1, \xi_2, \dots, \xi_m$. We add m new constraints (one for each data point) to disallow a negative slack (which would reward for points on the correct side of the margin). The scalar C is a hyper-parameter. It controls how to balance the size of margin against keeping points out of the margin.

Figure 7.6 shows the solution for $C = 100$ for a non-separable dataset. The decision boundary separates the all training points in two classes by the shown margin, except for two. The ξ_i slack variables for these two points are non-zero, therefore. One positive (orange +) point is on the wrong side of the boundary and so its ξ value is greater than 1. One negative (blue circle) point is within the margin, but still on the correct side of the boundary and so its ξ value is less than 1.

Loss Function

Equation 7.6 is the traditional specification of SVM learning. It is from this formulation that we will build a non-linear version. However, we can manipulate it to fit into the loss-minimization framework of previous chapters.

Further information

This is often called a “soft margin” formulation because we do not enforce the margin for all points, just encourage it.

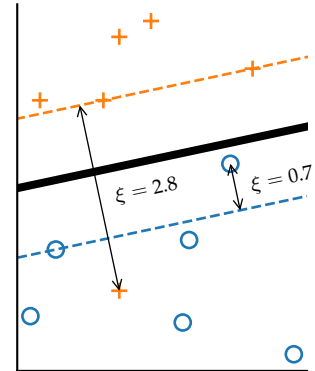


Figure 7.6: Decision boundary (solid) and margins (dashed). All but two slack variables, ξ s, are zero. The two non-zero slack variables are labeled.

The constraints for the slack variable for training point i can be rewritten as

$$\xi_i \geq 1 - y_i \tilde{f}(\vec{x}_i) \quad (7.7)$$

$$\xi_i \geq 0. \quad (7.8)$$

recall $\tilde{f}(\vec{x}_i) = \vec{x}_i^\top \vec{w} + b$

Because the optimization forces ξ_i to be as small as possible, we can conclude that at the solution ξ_i will be equal to whichever constraint is larger:

$$\xi_i = \max(0, 1 - y_i \tilde{f}(\vec{x}_i)). \quad (7.9)$$

Therefore SVMs seek to minimize

$$\frac{1}{2} \vec{w}^\top \vec{w} + C \sum_{i=1}^m \max(0, 1 - y_i \tilde{f}(\vec{x}_i)) \quad (7.10)$$

which is the same as minimizing

$$L = \sum_{i=1}^m l(y_i, \tilde{f}(\vec{x}_i)) + \lambda R(\vec{w}) \quad (7.11)$$

where

$$\begin{aligned} l(y, \tilde{f}) &= \max(0, 1 - y\tilde{f}) \\ R(\vec{w}) &= \vec{w}^\top \vec{w} \\ \lambda &= \frac{1}{2C}. \end{aligned} \quad (7.12)$$

This is a standard loss function with ℓ_2 regularization. The hyperparameter C in an SVM plays the inverse role of the regularization strength λ : Lower values of C result in more regularized (smooth) solutions. Higher values of C result in trying to fit the training data better.

The per-item loss for an SVM is known as the **hinge loss**. It and the loss functions for logistic regression and perceptron are plotted in Figure 7.7. While it is similar to the perceptron loss function, the shift to place the “hinge” at a positive value (and not at 0) is critical to it promoting large margins.

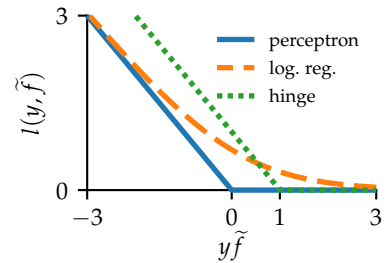


Figure 7.7: Comparison of hinge loss to other losses.

Dual Formulation

We can reformulate the optimization problem of Equation 7.6 to highlight the role of the support vectors and, later, create a non-linear version.

We start by writing the Lagrangian for Equation 7.6. There are m constraints for the margins, and we use a_i as the Lagrange multiplier

Key point

This part requires some understanding of Lagrange multipliers.

for the margin constraint on the i th data point. There are also m constraints on the slack variables, and we use r_i as the Lagrange multiplier for the i th slack variable. Denoting the Lagrangian as J ,

$$J = \underbrace{\frac{1}{2} \vec{w}^\top \vec{w}}_{\text{objective}} + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \overbrace{a_i (y_i (\vec{x}_i^\top \vec{w} + b) - 1 + \xi_i)}^{\text{enforcing the } i\text{th margin constraint}} - \underbrace{\sum_{i=1}^m r_i \xi_i}_{\text{enforcing slack variable } \xi_i \text{ be non-negative}}. \quad (7.13)$$

However, the constraints of Equation 7.6 are *inequality* constraints. Although we encode them with Lagrange multipliers, Lagrange multipliers enforce *equality* constraints. We solve this by forcing the Lagrange multipliers themselves to be positive. This ensures that the constraints are only enforced “on one side” of the inequality:²

² Why this works is beyond our scope.

$$\begin{aligned} a_i &\geq 0 && \text{for all } i \in \{1, \dots, m\} \\ r_i &\geq 0 && \text{for all } i \in \{1, \dots, m\} \end{aligned} \quad (7.14)$$

We can set the gradients of the Lagrangian with respect to the optimization variables equal to zero to find necessary conditions at the solution:

$$\begin{aligned} 0 = \nabla_{\vec{w}} J & & 0 = \frac{\partial J}{\partial b} & & 0 = \frac{\partial J}{\partial \xi_i} \\ & = \vec{w} - \sum_{i=1}^m a_i y_i \vec{x}_i & = \sum_{i=1}^m a_i y_i & & = C - a_i - r_i \\ \vec{w} = \sum_{i=1}^m a_i y_i \vec{x}_i & & 0 = \sum_{i=1}^m a_i y_i & & a_i = C - r_i \end{aligned} \quad (7.15)$$

The right-most condition, when coupled with the condition that $r_i \geq 0$, implies that $a_i \leq C$. We can then remove any further mention of r_i and simplify the inequalities of Equation 7.14 to

$$0 \leq a_i \leq C \quad \text{for all } i \in \{1, \dots, m\} \quad (7.16)$$

The left-most condition of Equation 7.15 states that the optimal weight vector can be written as a linear combination of the training data. Because a_i is always non-negative and y_i is the class label, this means the final solution has a positive contribution from positive examples and a negative contribution from negative examples.

We can simplify the Lagrangian with the equalities above that hold

at a solution:

$$\begin{aligned}
 J &= \frac{1}{2} \vec{w}^\top \vec{w} + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m a_i (y_i (\vec{x}_i^\top \vec{w} + b) - 1 + \xi_i) - \sum_{i=1}^m \overbrace{(C - a_i) \xi_i}^{r_i} \\
 &= \frac{1}{2} \vec{w}^\top \vec{w} + \underbrace{\sum_{i=1}^m C \xi_i}_{\text{these}} - \sum_{i=1}^m \overbrace{a_i y_i \vec{x}_i^\top \vec{w}}^{= \vec{w}^\top} - \sum_{i=1}^m \overbrace{a_i y_i b}^{= 0} + \sum_{i=1}^m a_i - \underbrace{\sum_{i=1}^m a_i \xi_i}_{\text{all}} - \underbrace{\sum_{i=1}^m (C - a_i) \xi_i}_{\text{cancel}} \\
 &= -\frac{1}{2} \vec{w}^\top \vec{w} + \sum_{i=1}^m a_i
 \end{aligned} \tag{7.17}$$

and then replace $\vec{w}$ with $\sum_{i=1}^m a_i y_i \vec{x}_i$

$$\begin{aligned}
 &= -\frac{1}{2} \left(\sum_{i=1}^m a_i y_i \vec{x}_i \right)^\top \left(\sum_{j=1}^m a_j y_j \vec{x}_j \right) + \sum_{i=1}^m a_i \\
 &= -\frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m a_i y_i \vec{x}_i^\top \vec{x}_j y_j a_j + \sum_{i=1}^m a_i \\
 &= -\frac{1}{2} \vec{a}^\top \mathbf{G} \vec{a} + \vec{a}^\top \vec{1}
 \end{aligned} \tag{7.18}$$

where $\vec{a}$ is the (m -dimensional) vector of the Lagrange multipliers, $\vec{1}$ is an m -dimensional vector of all 1s, and $\mathbf{G}$ is a (symmetric) matrix that depends only on the data.

The elements of $\mathbf{G}$ are

$$G_{ij} = y_i \vec{x}_i^\top \vec{x}_j y_j. \tag{7.19}$$

They are the inner products of two training points, multiplied by the corresponding class labels (+1 or -1). Thus, G_{ij} indicates how similar (or dissimilar) training points i and j are.

The Lagrangian corresponds to a solution when it is minimal with respect to the original variables ($\vec{w}, b, \xi_1, \dots, \xi_m$) and maximal with respect to the Lagrange multipliers ($a_1, \dots, a_m$). Our Lagrangian only has Lagrange multipliers left, so we can minimize its negation, leading to the following QP over the Lagrange multipliers of the original QP:

$$\begin{aligned}
 &\arg \min_{a_1, \dots, a_m} \quad \frac{1}{2} \vec{a}^\top \mathbf{G} \vec{a} - \vec{a}^\top \vec{1} \\
 &\text{s.t.} \quad 0 \leq a_i \leq C \quad \text{for all } i \in \{1, \dots, m\} \\
 &\quad \vec{a}^\top \mathbf{Y} = 0
 \end{aligned} \tag{7.20}$$

This is known as the dual QP. It is a minimization over variables that were Lagrange multipliers in the original (primal) QP (Equation 7.6).

Further information

Caution! $\vec{a}$ is an m -dimensional vector (one element per training example), unlike $\vec{x}$, which is an n -dimensional vector (one element per feature).

Properties of the Dual

Because the Lagrange multipliers (a_i and r_i) are only non-zero when the inequalities of Equation 7.6 are actually equalities (because otherwise, they do not need to be enforced), we can write

$$a_i (y_i (\vec{x}_i^\top \vec{w} + b) - 1 + \xi_i) = 0 \quad (7.21)$$

$$r_i \xi_i = 0. \quad (7.22)$$

These are a fancy way of writing an “or” statement. Because the multiplication of two values is zero, either the first or the second (or both) must be equal to zero. We are stating that either a Lagrange multiplier is zero, or the corresponding constraint is an equality.

Equation 7.21 tells us that if a training point is not on or within the margin (that is, $y_i (\vec{x}_i^\top \vec{w} + b) - 1 + \xi_i$ is not zero), then a_i must be 0. Therefore, the solution can be written in terms of only the points that are on the margin or on the wrong side of the margin. These are the **support vectors**: the training data points for which $a_i > 0$.

The dual program does not directly produce the vector $\vec{w}$ or offset b . Instead it produces coefficients $a_1, \dots, a_m$ from which $\vec{w}$ can be computed as $\vec{w} = \sum_{i=1}^m a_i y_i \vec{x}_i$. How to compute b is less obvious. We first note that from Equation 7.22 and Equation 7.15 we can conclude $(C - a_i)\xi_i = 0$. Thus, if $a_i < C$, then $\xi_i = 0$ and the point is not within the margin. Further, if $a_i > 0$ is also true, the point is exactly on the margin and $y_i (\vec{x}_i^\top \vec{w} + b) = 1$. We can rewrite this as

$$b = y_i - \vec{x}_i^\top \vec{w} \quad \text{for any } i \text{ such that } 0 < a_i < C. \quad (7.23) \quad \text{recall } y_i^2 \text{ is always 1}$$

So, we can pick any point whose a_i is not 0 or C and calculate b from that point. It should give the same answer for all such points (of which, there is guaranteed to be at least one). However, because the solution was found on a computer with finite precision arithmetic, it is better to take the average b so calculated across all such data points. If we let $\mathcal{M}$ be the set of indices of the points for which $0 < a_i < C$,

$$b = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} (y_i - \vec{x}_i^\top \vec{w}). \quad (7.24)$$

Algorithm 7.8 shows the complete dual learning algorithm.

While the primal and dual formulations provide the same answer, the dual formulation makes the support vectors more clear: They are the points for which $a_i > 0$. It also makes it clear that for many of the training points (perhaps most), $a_i = 0$.

This means that resulting weight vector, $\vec{w}$, is *sparse* in a particular sense. Typically, a sparse vector is one in which many of its elements are exactly equal to 0. That is not the case here. However, if instead

Further information

These are known as the (complementary slackness) Karush-Kuhn-Tucker conditions of the optimization problem.

SVM DUAL LEARNING

In: $\mathbf{X}, \mathbf{Y}, C$

Out: $\vec{w}, b$

$\mathbf{G} \leftarrow (\mathbf{Y}\mathbf{Y}^\top) \odot (\mathbf{X}\mathbf{X}^\top)$

$\vec{a} \leftarrow$

Solve-QP $\begin{pmatrix} \min \frac{1}{2} \vec{a}^\top \mathbf{G} \vec{a} - \vec{a}^\top \vec{1} \\ \text{s.t. } \vec{0} \leq \vec{a} \leq C \vec{1} \\ \vec{a}^\top \mathbf{Y} = 0 \end{pmatrix}$

$\vec{w} \leftarrow \sum_{i=1}^m a_i y_i \vec{x}_i$

$\mathcal{M} \leftarrow \{i \mid 0 < a_i < C\}$

$b \leftarrow \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} (y_i - \vec{x}_i^\top \vec{w})$

return $\vec{w}, b$

Algorithm 7.8: Learning the SVM binary classifier using the dual formulation. $\odot$ denotes element-wise multiplication.

of looking at $\vec{w}$ expressed in the standard basis (for which its elements are the coefficients), we express it in the basis of the training data vectors, the representation of $\vec{w}$ is sparse.

Sparsity is often prized in machine learning. It is useful computationally because it implies many operations do not need to be performed. But, it is also useful for generalization because statistical properties of the resulting decision function can be related to the sparsity of the solution theoretically. The details of why are beyond this text.

Non-Linear SVM

Although the dual formulation of the SVM QP is useful for demonstrating the solution's sparsity and support vectors, it is more helpful in constructing a non-linear version of SVMs.

Consider using the constructed features from Chapter 6. We can do this by replacing all instances of $\vec{x}_i$ with $\phi(\vec{x}_i)$. The function ϕ maps the n -dimensional vector of features of the original data into a p -dimensional vector. If this mapping is non-linear, linear classification in the new feature space is non-linear classification in the original feature space. The weight vector, $\vec{w}$, is now a p -dimensional vector.

Further information

We use p to denote the dimensionality of the constructed features

We must make only two changes to our dual formulation to use this feature mapping:

1. The dual QP (Equation 7.20) does not change, but the definition of the matrix G (Equation 7.19) does. Its elements are now

$$G_{ij} = y_i \phi(\vec{x}_i)^\top \phi(\vec{x}_j) y_j . \quad (7.25)$$

2. The resulting weight vector is now

$$\vec{w} = \sum_{i=1}^m a_i y_i \phi(\vec{x}_i) \quad (7.26)$$

which means that the discriminant is

$$\begin{aligned} \tilde{f}(\vec{x}) &= b + \left(\sum_{i=1}^m a_i y_i \phi(\vec{x}_i) \right)^\top \phi(\vec{x}) \\ &= b + \sum_{i=1}^m a_i y_i \phi(\vec{x}_i)^\top \phi(\vec{x}) . \end{aligned} \quad (7.27)$$

The Kernel Trick

Note that Equation 7.25 and Equation 7.27 only use ϕ when taking the inner product between two vectors in the constructed feature

space. We will call this inner product the **kernel** and denote it as

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^\top \phi(\vec{x}') . \quad (7.28)$$

The kernel takes two vectors in the original n -dimensional feature space and returns the inner product of their p -dimensional constructed feature vectors. We can now write

$$G_{ij} = y_i y_j k(\vec{x}_i, \vec{x}_j) \quad (7.29)$$

$$\tilde{f}(\vec{x}) = b + \sum_{i=1}^m a_i y_i k(\vec{x}_i, \vec{x}) . \quad (7.30)$$

Here the kernel is applied to two training points.

Here the kernel is applied to a training point and a testing point.

We cannot express the weight vector using only the kernel, but we can formulate the QP to find b and $a_1, \dots, a_m$ and, from these, predict the label for new points.

To see why removing ϕ and using only k matters, consider the following seven-dimensional constructed feature vector.

$$\phi \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_1x_2 \\ x_1^2 \\ x_2^2 \end{bmatrix} \quad (7.31)$$

For this feature mapping, the kernel would be

$$\begin{aligned} k(\vec{x}, \vec{x}') &= \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_1x_2 \\ x_1^2 \\ x_2^2 \end{bmatrix}^\top \begin{bmatrix} 1 \\ \sqrt{2}x'_1 \\ \sqrt{2}x'_2 \\ \sqrt{2}x'_1x'_2 \\ x_1'^2 \\ x_2'^2 \end{bmatrix} \\ &= 1 + 2x_1x'_1 + 2x_2x'_2 + 2x_1x'_1x_2x'_2 + x_1^2x_1'^2 + x_2^2x_2'^2 \\ &= (1 + x_1x'_1 + x_2x'_2)^2 \\ &= (1 + \vec{x}^\top \vec{x}')^2 . \end{aligned} \quad (7.32)$$

This kernel function can be computed without constructing the higher-dimensional features: We just take the inner product of the original two-dimensional vectors, add one, and then square it. The result is the same as constructing the seven-dimensional representations of both $\vec{x}$ and $\vec{x}'$ and then taking their inner product.

In this way, the kernel is more than just a notational shorthand for the inner product in the higher-dimensional space. The kernel is a function that is calculated without ever constructing $\phi(\vec{x})$. We will return to the power of using a kernel after detailing the full resulting algorithm.

Further information

An SVM is not the only machine learning method that can benefit from the kernel trick. Most other methods can be reformulated so that their learning algorithm and predictions depend on the training and testing data only through inner products. Then, those inner products can be replaced with a kernel function, thus turning a linear method into a non-linear method.

Algorithm

All together, the learning algorithm solves the QP

$$\begin{aligned} \arg \min_{a_1, \dots, a_m} \quad & \frac{1}{2} \vec{a}^\top \mathbf{G} \vec{a} + \vec{a}^\top \vec{1} \\ \text{s.t.} \quad & 0 \leq a_i \leq C \quad \text{for all } i \in \{1, \dots, m\} \\ & \vec{a}^\top \mathbf{Y} = 0 \end{aligned} \quad (7.33)$$

where

$$G_{ij} = y_i y_j k(\vec{x}_i, \vec{x}_j) . \quad (7.34)$$

which is the same as Equation 7.20 but with a different definition for $\mathbf{G}$. The result is not a weight vector, but an offset b and coefficients for each training point, $a_1, \dots, a_m$.

Prediction does not use a weight vector. Instead, it uses the learned coefficients $a_1, \dots, a_m$ and the corresponding training data:

$$\tilde{f}(\vec{x}) = b + \sum_{i=1}^m a_i y_i k(\vec{x}_i, \vec{x}) . \quad (7.35)$$

The testing point is $\vec{x}$. But, unlike previous classifiers we studied, this discriminant also depends (explicitly) on the training data: $\vec{x}_1, \dots, \vec{x}_m$ and $y_1, \dots, y_m$. The kernel function compares the testing point to a training point. Training points with greater kernel values have greater relevance to the final prediction because their a_i coefficients in the sum are multiplied by a larger value.

Algorithm 7.9 outlines the SVM learning algorithm when using a kernel. Prediction is done using Algorithm 7.10. Note that most of the coefficients, $a_1, \dots, a_m$ are 0 and therefore the corresponding training data points can be ignored. As shown, all of the training data $(\mathbf{X}, \mathbf{Y})$ must be passed into Algorithm 7.10. In practice, Algorithm 7.9 can record the just the coefficients for the training data with non-zero a_i values. These are the only points that need to be passed into the prediction.

Kernel Choice

We have traded the choice of a feature mapping, ϕ , for the choice of a kernel function, k . We can select any kernel function for which there exists some feature mapping so that Equation 7.28 is true: The kernel calculates the same value as mapping two data points with ϕ and then taking their inner product.

The simplest non-trivial kernel is just the inner product:

$$k(\vec{x}, \vec{x}') = \vec{x}^\top \vec{x}' . \quad (7.36)$$

KERNEL SVM LEARNING

In: $\mathbf{X}, \mathbf{Y}, C, k$

Out: $b, \vec{a}$

$\mathbf{K} \leftarrow$ matrix whose i - j element
is $k(\vec{x}_i, \vec{x}_j)$

$\mathbf{G} \leftarrow (\mathbf{Y}\mathbf{Y}^\top) \odot \mathbf{K}$

$\vec{a} \leftarrow$

Solve-QP $\begin{pmatrix} \min \frac{1}{2} \vec{a}^\top \mathbf{G} \vec{a} - \vec{a}^\top \vec{1} , \\ \text{s.t. } \vec{0} \leq \vec{a} \leq C \vec{1} \\ \vec{a}^\top \mathbf{Y} = 0 \end{pmatrix}$

$\mathcal{M} \leftarrow \{i \mid 0 < a_i < C\}$

$b \leftarrow$

$\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \left(y_i - y_i \sum_{j=1}^m y_j a_j k(\vec{x}_i, \vec{x}_j) \right)$

return $\vec{a}, b$

Algorithm 7.9: Learning the SVM binary classifier with a kernel. $\odot$ denotes element-wise multiplication.

KERNEL BINARY PREDICTION

In: $b, \vec{a}, \mathbf{X}, \mathbf{Y}, k, \vec{x}$

Out: $\tilde{f}(\vec{x})$

return $b + \sum_{i=1}^m a_i y_i k(\vec{x}_i, \vec{x})$

Algorithm 7.10: Prediction algorithm for an SVM with a kernel.

It corresponds to the feature map $\phi(\vec{x}) = \vec{x}$ and a linear classifier. The kernel measures the relevance between two points, or how much the label at one point relates to the label at another point. For this linear kernel, points that are in the same direction (small angle between the vectors $\vec{x}$ and $\vec{x}'$) have higher relevance. Also, points that are in the same direction, but further away from the origin have higher relevance. This might seem odd, but it is consistent with a linear classifier. If the point $\vec{x}$ likely has a positive label, then the point $2\vec{x}$ is even more likely to have a positive label. Linear functions extrapolate to more extreme values as they move away from the origin.

We can also construct kernels that correspond to polynomial feature maps. We already saw a quadratic kernel in Equation 7.32. More generally, a kernel that corresponds to polynomial features up through degree d can be formed as

$$k(\vec{x}, \vec{x}') = (c + \vec{x}^\top \vec{x}')^d \quad (7.37)$$

for any positive integer d and real value $c \geq 0$. Equation 7.32 is when $c = 1$ and $d = 2$. Assuming $c > 0$, this kernel corresponds to a $(n+1)^d$ -dimensional feature space. Again, this kernel assigns higher relevance to points that are in the same direction or farther away.

The second common kernel used with an SVM is the radial basis function (RBF) kernel:

$$k(\vec{x}, \vec{x}') = e^{-\gamma \|\vec{x} - \vec{x}'\|^2} = e^{-\gamma (\vec{x} - \vec{x}')^\top (\vec{x} - \vec{x}')}. \quad (7.38)$$

It has one hyper-parameter: γ . This kernel assigns higher relevance to points that are closer to each other ($\vec{x} - \vec{x}'$ is small). It assumes the farther two points are away, the less related the labels are. Surprisingly, this is both a valid kernel and corresponds to an infinite-dimensional feature space. How nice we do not have to explicitly construct $\phi(\vec{x})$!

These two kernels (the polynomial kernel and the RBF kernel) are the most frequently used. They each have hyper-parameters (c and d for the polynomial kernel and γ for the RBF kernel) which must be selected, usually with validation. The choice of the kernel itself is also a hyper-parameter and can also be selected with validation.

Other kernels are possible. For them to be valid, they must correspond to an inner product in some feature space. An advantage of using kernels is that they can be formed for examples that are not vectors. For instance, given a dataset of DNA fragments (strings of differing lengths) from hippopotamuses, we could employ the substring kernel which counts the number of substrings in common between the two sequences (using efficient data structures). This would allow classification of hippopotamuses based on DNA fragments without the explicit construction of any feature vectors.

Further information

The RBF kernel is also known as the squared exponential kernel or the Gaussian (RBF) kernel. Note the use of the term “Gaussian” merely notes that the mathematical form is similar to that of a Gaussian random variable’s probability density function. It is not because there is a normal or Gaussian random variable involved. The term “radial basis function” comes from a particular mathematical form of a neural network. An SVM takes on the same form when using this kernel.

Further information

How can a kernel correspond to an infinite-dimensional feature space and still be computable? If the components of this feature space just happen to line up nicely, their infinite sum (series) has a nice closed-form solution. That works out for the RBF kernel, as an example.

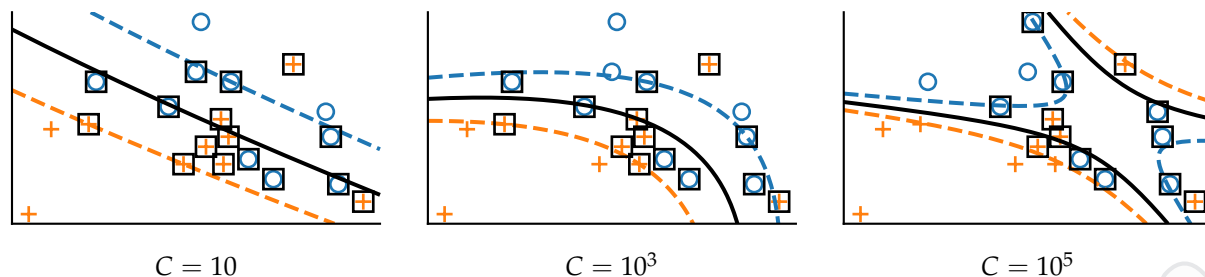


Figure 7.11: Examples of learned SVMs with a polynomial kernel with $d = 2$ and $c = 1$: $k(\vec{x}, \vec{x}') = (1 + \vec{x}^\top \vec{x}')^2$. The margin violation penalty, C , is the only difference among the plots. Boxes are drawn around the support vectors.

SVM Kernel Examples

Figure 7.11 shows the result of binary classification using SVMs and a polynomial (quadratic) kernel. Figure 7.12 shows the same for an RBF kernel. The hyper-parameter C sets how important it is to ensure all points are on the correct side of the boundary and outside of the margin. It has the inverse effect of the regularization strength from Chapter 6. Low values of C correspond to high regularization and the decision surface tends to be simpler. If C is too small, this can result in underfitting. High values of C correspond to low regularization and the decision surface tends to be more complicated. If C is too large, this can result in overfitting. In these examples, it is not clear when either underfitting or overfitting has happened. With real datasets, cross-validation would be used to pick C .

The margins (shown as dashed lines) are constant (and equal) distances from the decision boundaries (shown as solid lines). However, this is in the feature space, where the SVM has a linear decision surface. When projected back to the two-dimensional space of the

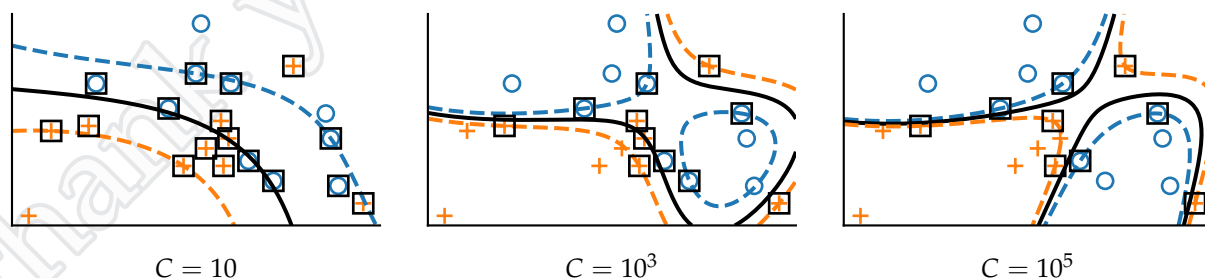


Figure 7.12: Examples of learned support vector machines with an RBF kernel with $\gamma = 1$: $k(\vec{x}, \vec{x}') = e^{-\|\vec{x} - \vec{x}'\|^2}$. The margin violation penalty, C , is the only difference among the plots. Boxes are drawn around the support vectors.

original data, the geometric margin varies. The support vectors are those where the corresponding a_i values are non-zero. These are also the points that are either exactly on the margin (if $a_i < C$) or within the margin. In general, when C is larger, there are fewer support vectors.

thank you for not distributing

Exercises

Exercise 7.1

Consider the dataset in Table 7.13.

- For the weight vector $\vec{w} = \begin{bmatrix} -3 \\ 1 \end{bmatrix}$ and intercept term $b = 3$, what is the *geometric margin* of this dataset?
- For the weight vector $\vec{w} = \begin{bmatrix} -3 \\ 1 \end{bmatrix}$ and intercept term $b = 3$, what is the *functional margin* of this dataset?
- For the weight vector $\vec{w} = \begin{bmatrix} -6 \\ 2 \end{bmatrix}$ and intercept term $b = 6$, what is the *geometric margin* of this dataset?
- For the weight vector $\vec{w} = \begin{bmatrix} -6 \\ 2 \end{bmatrix}$ and intercept term $b = 6$, what is the *functional margin* of this dataset?
- What is the maximum geometric margin for this dataset? (You should be able to determine this by plotting the data.)
- What are the values for $\vec{w}$ and b for this maximum margin boundary, assuming the functional margin is 1?

Exercise 7.2

Consider the dataset in Table 7.14.

- For the weight vector $\vec{w} = \begin{bmatrix} -2 \\ -1 \\ 3 \end{bmatrix}$ and intercept term $b = -5$, what is the *geometric margin* of this dataset?
- For the weight vector $\vec{w} = \begin{bmatrix} -2 \\ -1 \\ 3 \end{bmatrix}$ and intercept term $b = -5$, what is the *functional margin* of this dataset?
- The maximum margin hyperplane for this dataset has $w_2 = -1/3$ and $w_3 = 2/3$. Thus, the SVM solution is of the form

$$\vec{w} = \begin{bmatrix} w_1 \\ -1/3 \\ 2/3 \end{bmatrix}$$

$$b = b$$

What are the values for w_1 and b ?

Hint: The problem is reduced to only two degrees of freedom: w_1 and b . Plot the constraints on these two variables from Equation 7.5 and visually inspect it to find the solution.

Exercise 7.3

For each part below, the same dataset has been fit with a SVM with the same kernel three times. Each time a different value of C was used. For each part, indicate which decision surface was fit with the smallest C value and which was fit with the largest C value, and why.

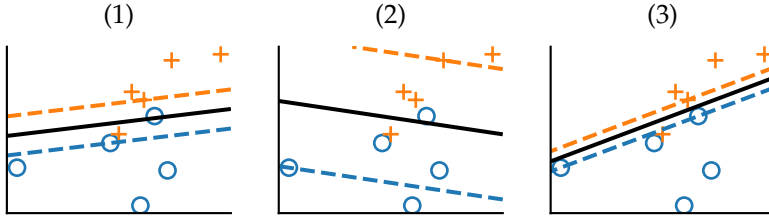
x_1	x_2	y
-1	0	+1
0	1	+1
3	2	-1
4	0	-1
3	-1	-1

Table 7.13: Toy margin dataset.

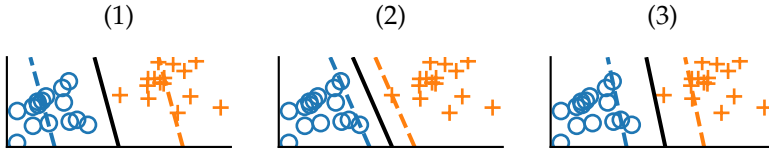
x_1	x_2	x_3	y
-1	-1	3	+1
1	-1	3	+1
-1	0	4	+1
0	0	2	+1
1	2	0	-1
1	1	0	-1

Table 7.14: Second toy margin dataset.

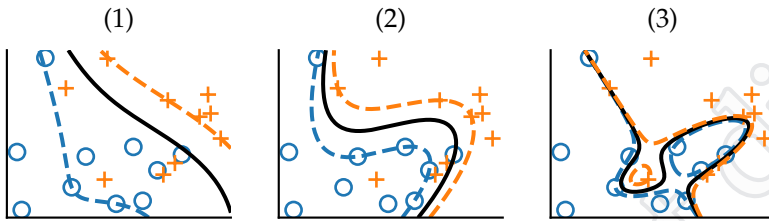
a. Using a linear kernel: $k(\vec{x}, \vec{x}') = \vec{x}^\top \vec{x}'$



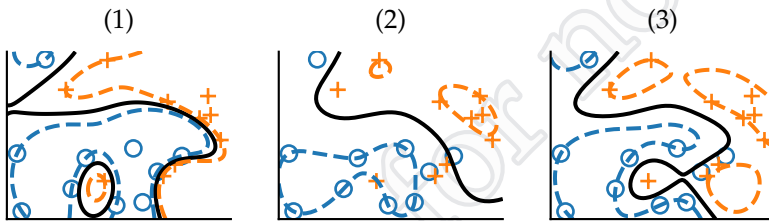
b. Using a linear kernel: $k(\vec{x}, \vec{x}') = \vec{x}^\top \vec{x}'$



c. Using a cubic kernel: $k(\vec{x}, \vec{x}') = (\vec{x}^\top \vec{x}' + 1)^3$



d. Using an RBF kernel: $k(\vec{x}, \vec{x}') = e^{-(\vec{x} - \vec{x}')^\top (\vec{x} - \vec{x}')}$



Exercise 7.4

Although convergence can be difficult, gradient descent for SVM is possible.

- Derive the gradient descent update rule for the primal (non-kernel) SVM formulation. The loss function is as in Equation 7.11.
- What is the stochastic gradient descent update rule? (You will need to incorporate the regularizer term into the sum over examples.)
- Contrast this to the perceptron learning algorithm. Equation 7.11.

Exercise 7.5

Consider penalizing the slack variables, ξ_i , by their square. This is

called an L2-SVM (in contrast to an L1-SVM). That is, in Equation 7.6, replace $C \sum_{i=1}^m \xi_i$ with $\frac{C}{2} \sum_{i=1}^m \xi_i^2$. (Using $\frac{C}{2}$ as the constant instead of C is traditional for this formulation and makes the derivations a little easier.)

- Explain why we can then drop the $\xi_i \geq 0$ constraint from the program. Write down the full (primal) QP.
- What is the per-item loss for this L2-SVM (called the squared hinge loss)? Write down an explicit formula for $l(y, \tilde{f})$ and plot it in comparison with the hinge loss.
- What is the dual formulation for an L2-SVM? That is, derive the equivalent to Equation 7.20, but for an L2-SVM.

Exercise 7.6

- Generalize the feature vector from Exercise 6.8 part b to higher dimensions. That is, if the raw features were of dimension n , what is the feature vector that allows linear classification in the constructed feature space to correspond to a circle/sphere/hyper-sphere in the raw feature space?
- What is the kernel that corresponds to this feature space?

Exercise 7.7

We have surveyed hippopotamuses to find out how much time they spend in various activities (sleeping, eating, playing hide-and-seek, etc.). In total, there are 10 such activities and each hippopotamus is described by a 10-dimensional vector whose elements are the fraction of the day it spends on each activity. (Thus, the features sum to 1 and are all between 0 and 1.) We would like to build a classifier to predict which hippopotamuses will buy our new sunscreen.

- Consider using the infinite-dimensional feature map

$$\phi(\vec{x}) = \left[1 \quad x_1 \quad x_2 \quad \dots \quad x_{10} \quad x_1^2 \quad x_2^2 \quad \dots \quad x_{10}^2 \quad x_1^3 \quad x_2^3 \quad \dots \quad x_{10}^3 \quad \dots \right]^T$$

Show that we can calculate the inner product (that is, the kernel) between two such feature vectors without constructing the entire (infinite) feature vectors. What is the resulting kernel?

- Consider using the squared exponential kernel with $\gamma = 1$: $k(\vec{x}, \vec{x}') = e^{-(\vec{x} - \vec{x}')^T(\vec{x} - \vec{x}')}$. Demonstrate this corresponds to a (different) infinite-dimensional feature mapping, $\phi(\vec{x})$. Hint: You may want to consider the Taylor expansion of e^x .

Exercise 7.8

Demonstrate that logistic regression (without regularization) can also be done with kernels (that is, “kernelized”).

- Explain why the logistic regression solution (without regu-

larization and without kernels) found when gradient descent starts from $\vec{w} = \vec{0}$ can be expressed as

$$\vec{w} = \sum_{h=1}^m a_h y_h \vec{x}_h$$

where a_h is a non-negative scalar.

- b. What is the (batch) gradient descent rule for logistic regression when parameterized in terms of $a_1, \dots, a_m$ instead of $\vec{w}$?
- c. Reformulate this update rule so that it no longer refers to the vector $\vec{x}_1, \dots, \vec{x}_m$ directly, and instead only refers to them through the kernel k , representing the inner product of two feature vectors.
- d. After learning with a kernel k , what is the prediction function, $\tilde{f}(\vec{x})$?